

# AMM Turbine v1 Flow Diagram

What an AMM is, how liquidity becomes LP tokens, and how withdraw/swap flows work

## What is an AMM?

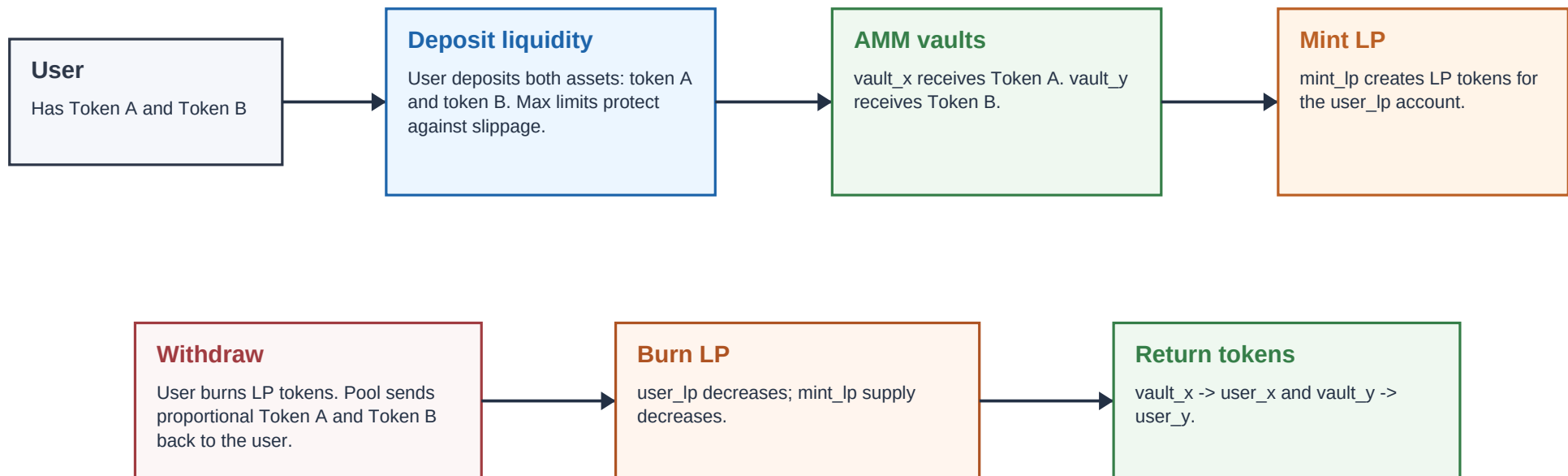
An Automated Market Maker is a pool of two tokens. Traders swap against the pool, and liquidity providers own a share of the pool through LP tokens.

## Core idea

The vaults hold Token A and Token B. The constant-product curve uses  $x * y = k$  to price swaps and preserve pool balance.

## LP token

An LP token is a receipt for pool ownership. In the first deposit example: Token A + Token B  $\Rightarrow$  1 LP share unit.

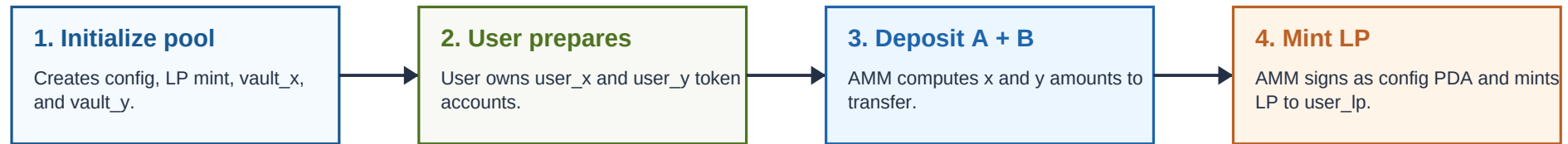


## Mental model:

Deposit adds balanced liquidity and mints ownership. Withdraw burns ownership and returns the underlying tokens. Swap trades one token for the other through the pool while the curve updates the effective balances.

# Liquidity Provider Flow

Initialize, deposit Token A + Token B, then mint LP tokens



## Deposit formula used by the program

### First liquidity

If `mint_lp.supply == 0` and both vaults are empty, the program uses `max_x` and `max_y` as the starting liquidity amounts.

### Existing pool

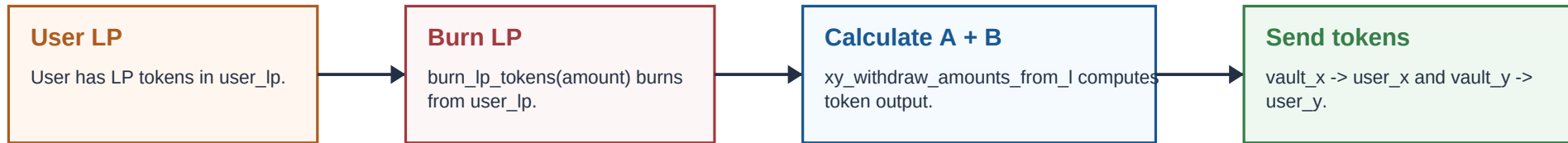
For later deposits, `ConstantProduct::xy_deposit_amounts_from_l(vault_x, vault_y, lp_supply, amount, precision)` computes the proportional token amounts.

| Instruction                                | Inputs                                                                                          | What changes                                                                                                                                 |
|--------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>deposit(amount, max_x, max_y)</code> | <code>amount</code> = LP tokens user wants; <code>max_x/max_y</code> = upper token spend limits | <code>user_x</code> and <code>user_y</code> decrease; <code>vault_x</code> and <code>vault_y</code> increase; <code>user_lp</code> increases |
| LP relationship                            | Example mental model: (Token A, Token B) => 1 LP share                                          | LP tracks ownership of the pool rather than being Token A or Token B itself                                                                  |
| Slippage guard                             | <code>amounts.x &lt;= max_x</code> and <code>amounts.y &lt;= max_y</code>                       | Deposit fails if required token amounts exceed the user's limits                                                                             |

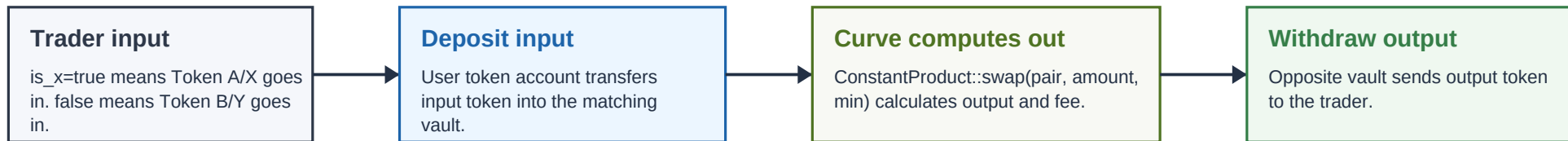
# Withdraw and Swap Flow

Burn LP to exit liquidity; swap one token for the other through the vaults

## Withdraw path



## Swap path



| Flow                                  | Inputs                                                                           | Guard                                         |
|---------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------|
| withdraw(amount, min_x, min_y)        | amount = LP to burn; min_x/min_y = minimum tokens to receive                     | Fails if x < min_x or y < min_y               |
| swap(is_x, amount_in, min_amount_out) | is_x chooses input side; amount_in is input; min_amount_out is output protection | Fails if curve output is below min_amount_out |

# Accounts and Structs Needed

Anchor account structs and the token accounts each instruction needs

| Config field          | Meaning                                            |
|-----------------------|----------------------------------------------------|
| seed                  | Pool seed used to derive the config PDA.           |
| authority             | Optional authority that can control pool settings. |
| mint_x / mint_y       | Token A and Token B mints for this pool.           |
| fee                   | Swap fee in basis points.                          |
| locked                | If true, deposit/withdraw/swap should be blocked.  |
| config_bump / lp_bump | Bumps for config PDA and LP mint PDA.              |

| Instruction struct | Needed accounts                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Initialize         | initializer signer, mint_x, mint_y, mint_lp PDA, vault_x ATA, vault_y ATA, config PDA, token_program, associated_token_program, system_program   |
| Deposit            | user signer, mint_x, mint_y, config, mint_lp, vault_x, vault_y, user_x, user_y, user_lp, token_program, system_program, associated_token_program |

### PDA and token account rules

config PDA: seeds ["config", seed]. LP mint PDA: seeds ["lp", config.key()]. Vault token accounts are ATAs owned by config. User token accounts are ATAs owned by user.

ser\_x,

ser\_x,

### In plain English

Liquidity providers deposit both tokens and receive LP. To exit, they burn LP and receive both tokens back. Swappers deposit one token and receive the other token from the pool.